Sprog og oversættere

Lexical analysis: Regular expressions and NFAs

Part 3: Regular expressions

Hans Hüttel

(using content by Cosmin Oancea)

The regular operators

The regular operators are three operators that we can use to build new languages from existing ones.

Definition

Suppose L_1 and L_2 are languages. Then

- The union of L_1 and L_2 is

$$L_1 \cup L_2 \stackrel{\text{def}}{=} \{w \mid w \in L_1 \text{ or } w \in L_2\}$$

- The concatenation of L_1 and L_2 is

$$L_1 \circ L_2 \stackrel{\text{def}}{=} \{uv \mid u \in L_1 \text{ and } v \in L_2\}$$

- The star of L_1 is

$\varepsilon \in L_1^*$ (sequence with 0 strings from L_1)

$$L_1^* = \{w_1 \dots w_k \mid k \geq 0, w_i \in L_1 \text{ for all } 1 \leq i \leq k\}$$

Examples of the use of the regular operators

Suppose

$$L_1 = \{a, b\}$$
$$L_2 = \{c, d\}$$

Then

$$L_1 \circ L_2 = \{ac, ad, bc, bd\}$$

We have that

$$L_1^* = \{\varepsilon, a, b, aa, ab, ba, bb, aab \ldots\}$$

Note that for any non-empty language L, L^* contains infinitely many strings.

$L = \emptyset \Rightarrow L^* = \{\varepsilon\}$

Regular expressions

Regular expressions are a formal notation that allows us the describe the structure of tokens using the regular operators.

Definition

The set $RE(\Sigma)$ of regular expressions over alphabet Σ is defined by

- Basic expressions
  - $\epsilon \in RE(\Sigma)$
  - $a \in RE(\Sigma)$ for every $a \in \Sigma$

Regular expressions

Regular expressions are a formal notation that allows us the describe the structure of tokens using the regular operators.

Definition

The set $RE(\Sigma)$ of regular expressions over alphabet Σ is defined by

- Basic expressions
  - $\epsilon \in RE(\Sigma)$
  - $a \in RE(\Sigma)$ for every $a \in \Sigma$
- Composite expressions:
  - $R_1 \circ R_2 \in RE(\Sigma)$

Regular expressions

Regular expressions are a formal notation that allows us the describe the structure of tokens using the regular operators.

Definition

The set $RE(\Sigma)$ of regular expressions over alphabet Σ is defined by

- Basic expressions
  - $\epsilon \in RE(\Sigma)$
  - $a \in RE(\Sigma)$ for every $a \in \Sigma$
- Composite expressions:
  - $R_1 \circ R_2 \in RE(\Sigma)$
  - $R_1 \mid R_2 \in RE(\Sigma)$

Regular expressions

Regular expressions are a formal notation that allows us the describe the structure of tokens using the regular operators.

Definition

The set $RE(\Sigma)$ of regular expressions over alphabet Σ is defined by

- Basic expressions
  - $\epsilon \in RE(\Sigma)$
  - $a \in RE(\Sigma)$ for every $a \in \Sigma$
- Composite expressions:
  - $R_1 \circ R_2 \in RE(\Sigma)$
  - $R_1 \mid R_2 \in RE(\Sigma)$
  - $R_1* \in RE(\Sigma)$

Some conventions

- Often we leave out $\circ$: $a \circ b = ab$
- We can use parentheses if we want to
- $\circ$ binds tighter than $|$: $a|bc* = a|(b(c*))$.
- $*$ binds tigher than $\circ$ and $|$

Every regular expression defines a language

Every regular expression R defines a language $L(R)$.

We can define this in the structure of R

Definition

$$L(a) = \{a\}$$
$$L(\varepsilon) = \{\epsilon\}$$
$$L(R_1 \mid R_2) = L(R_1) \cup L(R_2)$$
$$L(R_1 \circ R_2) = L(R_1) \circ L(R_2)$$
$$L(R*) = (L(R))^*$$

Note: This is the definition used in *Syntax and semantics* course.

We give a different (but equivalent) definition here that uses rewrite rules.

Comparing different notations

The notation for used in this course is different from the one used elsewhere (supposedly more keyboard-friendly).

Notation used here	**Name**	**Rewrite rules**	**In Sipser's book**
a	Symbol	None	a
ε	The language of the empty string	None	ε
Not used!	The empty language	None	$\emptyset$

Comparing different notations

The notation for used in this course is different from the one used elsewhere (supposedly more keyboard-friendly).

Notation used here	**Name**	**Rewrite rules**	**In Sipser's book**
$r_1 \mid r_2$	Choice	$r_1 \mid r_2 \to r_1$ $r_1 \mid r_2 \to r_2$	$r_1 \cup r_2$
$r_1 r_2$	Concatenation	$r_1 r_2 \to r_1' r_2$ whenever $r_1 \to r_1'$	$r_1 \circ r_2$
$r*$	Star	$r* \to r r*$ $r^* \to \varepsilon$	r^*

Using the rewrite rules to build strings

Suppose we have the regular expression

$$(a|aab) * (baa)$$

What strings can we build from it?

$r_1 r_2 \longrightarrow r_2' r_2$

if $r_1 \longrightarrow r_2'$

$r^* \longrightarrow r r^*$

$r^* \longrightarrow \varepsilon$

$r_1 | r_2 \longrightarrow r_1$

$r_1 | r_2 \longrightarrow r_2$

$(a|aab)^* \longrightarrow (a|aab)(a|aab)^*$

$\longrightarrow aab\,(a|aab)^* \longrightarrow aab\,\varepsilon \approx aab$

$\longrightarrow aab \longrightarrow aab \longrightarrow aab$

$(baa) \longrightarrow baa \longrightarrow baa \longrightarrow baa$

$(a|aab)^*(baa) \longrightarrow \cdots \longrightarrow aabbaa$

Useful abbreviations for regular expressions

- Character sets:

$$a_1 \mid a_2 \mid \cdots \mid a_n \stackrel{\text{def}}{=} [a_1 a_2 \ldots a_n]$$

- Negation: $\hat{}R$ describes any string *not* described by R
- Character ranges:

$$[a_1 - a_n] \stackrel{\text{def}}{=} (a_1 \mid a_2 \mid \cdots a_n)$$

  where the characters a_i are ordered

- Optional parts:

$$R? \stackrel{\text{def}}{=} (R \mid \varepsilon)$$

- Repetition:

$$R+ \stackrel{\text{def}}{=} (R* \mid R)$$

  – strings composed of one or more strings described by R

A regular expression for integer numerals

Integers in decimal format: `234`, `0`, `8` but not `08` or `abc`!

can be described by

$$(1 \mid 2 \mid \ldots \mid 9)(0 \mid 1 \mid 2 \mid \ldots \mid 9)* \mid 0$$

A regular expression for integer numerals

Integers in decimal format: `234`, `0`, `8` but not `08` or `abc`!

can be described by

$$(1 \mid 2 \mid \ldots \mid 9)(0 \mid 1 \mid 2 \mid \ldots \mid 9)* \mid 0$$

If we use the character range shorthand (`[-]`) we get

$$[1-9][0-9]* \mid 0$$

A regular expression for hexadecimal integers

Integers in hexadecimal format: `0X123`, `0xcafe` but not `0X`, `0XG`!

can be described by

$$0(x \mid X)[0-9a-fA-F][0-9a-fA-F]*$$

A regular expression for hexadecimal integers

Integers in hexadecimal format: `0X123`, `0xcafe` but not `0X`, `0XG`!

can be described by

$$0(x \mid X)[0-9a-fA-F][0-9a-fA-F]*$$

If we use the shorthand for "at least one" $(+)$ we get

$$0(x \mid X)[0-9a-fA-F]+$$

A regular expression describing variable names

A variable name consists of letters, digits and underscore, and it must begin with a letter or underscore.

This can be described by

$$[a-zA-Z\_][a-zA-Z\_0-9]*$$